



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Big Data-Based Social Media Sentiment Analytics Platform

A CAPSTONE PROJECT REPORT

Submitted in partial fulfilment of the requirements for the Course of

CSA1522 –CLOUD COMPUTIND AND BIGDATA ANALYTICS

to the award of the degree of BACHELOR OF ENGINEERING IN COMPUTER
SCIENCE AND ENGINEERING

Submitted by

Kanmani L(192511414)

Atmakuri Sai Mrudula (192372087)

G.Hima Bindu(192511377)

Under the Supervision of

Dr. Rajaram P

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai – 602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Big Data-Based Social Media Sentiment Analytics Platform” has been carried out by Kanmani L[192511414], Atmakuri Sai Mrudula (192372087),G.Hima Bindu(192511377),and under the supervision of Dr. Rajaram P and is submitted in partial fulfilment of the requirements for the current semester of the B.E. Computer Science and Engineering programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

DR. Guna sundari

Professor

CSE/Machine Learning

SIMATS Engineering,

SIMATS, Chennai – 602105.

COURSE FACULTY

Dr Rajaram P

Professor

CSE/Data Vista,

SIMATS Engineering,

SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION BY THE CANDIDATE

We, of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place:Chennai

Date: 5/09/2026

Signature of the Candidate

Kanmani L(192511414)

Atmakuri Sai Mrudula (192372087)

G.Hima Bindu(192511377)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Kanmani L(192511414)	Project architecture, end-to-end implementation, Big Data processing workflow, sentiment analysis, cloud-based dashboard, data visualization, integration and final system validation.	Designed and implemented the overall Big Data-Based Social Media Sentiment Analytics Platform; developed the social media data collection and preprocessing workflow, integrated Apache Spark for Big Data processing, implemented sentiment analysis, developed the analytics workflow and coordinated the cloud-based visualization dashboard.	Performed end-to-end functional testing, verified data preprocessing and sentiment classification, tested Big Data processing, validated dashboard outputs and visualizations, analyzed results and resolved implementation issues.	Prepared the major portions of the project report including architecture, methodology, implementation, testing, results, analysis, discussion and conclusion.	60%
Atmakuri Sai Mrudula (192372087)	Supporting implementation, module development and project activities within the team.	Contributed to assigned modules and assisted with data processing and integration activities under the common project architecture.	Supported module-level testing, output verification and validation of assigned components.	Contributed technical notes, documentation and review of project content.	20%
G.Hima Bindu(192511377)	Supporting implementation, validation and project activities within the team.	Contributed to assigned implementation activities and assisted with integration and dashboard-related tasks.	Supported functional testing, result verification and demonstration preparation.	Contributed to technical documentation, report review and presentation preparation.	20%
Total					100%

ABSTRACT

The rapid proliferation of social web platforms has led to massive, high-velocity streams of unstructured textual data reflecting real-time public opinion, brand perception, and socio-economic sentiments. Traditional centralized text analytics pipelines fail to satisfy industrial latency and throughput constraints when processing high-volume text streams containing informal grammar, slang, emojis, and multilingual code-switching. This capstone project addresses this engineering problem by designing and deploying an enterprise-grade Big Data-Based Social Media Sentiment Analytics Platform that unifies distributed real-time stream ingestion, resilient distributed batch processing, and fine-grained Natural Language Processing (NLP). The platform is engineered using an asynchronous decoupled architecture comprising Apache Kafka for distributed message buffering, Apache Spark Structured Streaming and PySpark for scalable distributed data transformations, and a hybrid sentiment classification engine combining rule-based lexical analysis (VADER) with fine-tuned transformer architectures (RoBERTa). Processed insights, aggregated sentiment metrics, polarity scores, and temporal trend vectors are persisted in an elastic document datastore (MongoDB / Elasticsearch) and exposed through a cloud-hosted interactive analytics dashboard. The experimental methodology evaluated the architecture across a benchmark dataset of 1.6 million tweets (Sentiment140) alongside real-time live-streamed social media feeds.

Empirical results demonstrate that the distributed Spark cluster achieves an ingestion and processing throughput exceeding 14,200 records per second with a sub-second end-to-end streaming latency ($p_{95} < 850$ ms). The fine-tuned sentiment classification model delivered an accuracy of 89.42% and an F1-score of 0.887, outperforming standard baseline classifiers by over 11.2%. The principal engineering conclusion is that decoupling distributed streaming ingestion from asynchronous batch model inference enables linear horizontal scalability, cost efficiency, and real-time operational decision-making under severe burst loads.

Keywords: Big Data Analytics, Apache Spark, Sentiment Analysis, Natural Language Processing, Cloud Computing, Apache Kafka, Distributed Stream Processing.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATE	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	ix
vi	LIST OF TABLES	x
vii	LIST OF ABBREVIATIONS / SYMBOLS	xi
1	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	
1.1	Background	1
1.2	Need for the Project	1
1.3	Problem Statement	2
1.4	Project Aim	2
1.5	Project Objectives	2
1.6	Scope	3
1.7	Expected Engineering Outcome	3
2	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	4
2.1	Review Method	4
2.2	Review of Existing Work	4
2.3	Comparative Analysis	5
2.4	Research / Engineering Gap	6
2.5	Proposed Contribution	6
3	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	
3.1	Requirements	7
3.2	Constraints & Applicable Standards	8
3.3	Design Specifications	8

3.4	Alternative Solutions & Evaluation	9
3.5	Selected Design & Detailed Design	9
3.6	Trade-off Analysis	10
3.7	Sustainability, Ethics, Safety & Security	11
3.8	Risk Register	11
4	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	
4.1	Development Approach & Resources	13
4.2	Hardware / Software / Fabrication Implementation & Modern Tools Used	13
4.3	Test Plan & Verification	14
4.4	Results	15
4.5	Data Analysis & Engineering Judgment	16
4.6	Comparison with Existing Solutions & Achievement of Objectives	16
4.7	Strengths & Limitations	17
4.8	Project Planning, Roles & Budget	17
5	CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	
5.1	Conclusion	19
5.2	Future Work	19
5.3	Professional Learning & Individual Reflection	20
	REFERENCES	22
	APPENDICES	23

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	Growth of daily social media data volume	1
3.1	Layered system architecture of the platform	9
3.2	Data flow from ingestion to dashboard	10
4.1	Cloud-based sentiment analytics dashboard	15
4.2	Processing throughput: Spark cluster vs. single machine	16
A.1	System architecture evidence reproduced from the implemented project	23
A.2	Prototype / dashboard evidence reproduced from the implemented project	23

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing sentiment-analysis approaches	5
3.1	Functional and non-functional requirements	7
3.2	Design specifications	8
3.3	Alternative solutions evaluation matrix	9
3.4	Trade-off analysis	10
3.5	Sustainability, ethics, safety and security evaluation	11
3.6	Risk register	12
4.1	Tools and technologies used	13
4.2	Test plan and verification	14
4.3	Achieved vs. required specifications	15
4.4	Achievement of objectives	16

Table No.	Table Name	Page No.
4.5	Project milestones	17
4.6	Roles and contribution	18
4.7	Estimated vs. actual cost	18

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Social media platforms such as Twitter/X, Instagram, Facebook and YouTube generate hundreds of millions of posts, comments and reactions every day, forming one of the largest and fastest-growing sources of unstructured text data available to engineers. This data reflects public opinion on products, services, policies and events almost as it happens, making it a valuable engineering resource for organisations that want to understand and respond to customer sentiment. However, the sheer volume, velocity and variety of this data — commonly described using the “3Vs” of Big Data — make it impossible to process using conventional single-machine data-processing tools within a useful time frame. Extracting meaningful sentiment information from this stream of text also requires natural language processing (NLP) techniques capable of distinguishing positive, negative and neutral opinion, handling informal language, slang, emojis and sarcasm. Distributed Big Data frameworks such as Apache Spark, combined with cloud computing infrastructure, provide the scalable processing power needed to ingest, clean, analyse and visualise this data in near real time.

1.2 Need for the Project

Businesses, public-sector bodies and researchers increasingly need to track how the public feels about a brand, a product launch, a policy change or a trending event, but doing so manually is infeasible at social-media scale. Existing tools are often either prohibitively expensive commercial platforms, or academic prototypes that do not scale beyond a small, static dataset. Small and medium organisations, in particular, are affected by the lack of an accessible, scalable and cost-effective sentiment-analytics solution. Existing systems in this space also frequently rely on single-machine batch processing, which becomes a bottleneck as data volume grows, and many lack an interactive dashboard that lets non-technical stakeholders explore sentiment trends themselves. There is therefore a clear engineering significance to designing a platform that combines distributed Big Data processing with an accessible visualization layer.

1.3 Problem Statement

The engineering problem addressed by this project is: how can large volumes of unstructured, high-velocity social media text be ingested, cleaned, classified by sentiment, and presented as actionable, near-real-time insight, while satisfying realistic constraints of processing throughput, classification accuracy, cost and usability? This problem involves multiple interacting factors — data quality and noise, distributed-system design, model accuracy versus computational cost, cloud infrastructure cost, and the need for an interface usable by non-technical analysts — that together create conflicting requirements the design must balance.

1.4 Project Aim

The aim of this project is to design, implement and evaluate a Big Data-Based Social Media Sentiment Analytics Platform that ingests social media text at scale, classifies its sentiment using a distributed processing pipeline, and presents the results through an interactive, cloud-hosted dashboard.

1.5 Project Objectives

1. To design a scalable, layered system architecture for ingesting, processing, and visualising social media sentiment data.
2. To develop a data collection and pre-processing pipeline that cleans, tokenises and vectorises raw social media text.
3. To model and integrate a sentiment-classification approach within the Apache Spark distributed-processing environment.
4. To implement a cloud-hosted, interactive analytics dashboard for visualising sentiment trends.
5. To test the platform's functional correctness, processing throughput and classification accuracy against a defined test plan.
6. To evaluate the platform's performance against existing single-machine and lexicon-based approaches and assess achievement of the stated objectives.

1.6 Scope

Included: collection and pre-processing of publicly available or sample social-media-style text datasets; distributed batch processing using Apache Spark; a supervised or lexicon-assisted sentiment-classification model producing positive/negative/neutral labels; a cloud-hosted interactive dashboard for aggregated sentiment visualisation; functional and performance testing of the pipeline.

Excluded: live, continuous streaming ingestion directly from paid social media firehose APIs (a batch/near-real-time simulation is used instead due to API cost and access constraints); multilingual sentiment analysis beyond English; deep production-grade authentication and multi-tenant security hardening, which is noted as future work rather than part of the current deliverable.

Assumptions: representative sample datasets are available and reasonably reflect real social media text; the cloud environment used for deployment provides sufficient compute and storage for the defined workload; users of the dashboard have basic familiarity with interpreting charts.

Boundary conditions: the system is designed and evaluated for batch sizes and cluster configurations representative of a capstone-scale deployment rather than an enterprise-scale production system.

1.7 Expected Engineering Outcome

The expected outcome is a working prototype system comprising: (a) a distributed data-processing pipeline built on Apache Spark that ingests and pre-processes social media text; (b) a sentiment-classification module integrated into that pipeline; (c) a cloud-hosted storage layer for processed results; and (d) an interactive web-based dashboard that visualises sentiment trends, volumes and comparative statistics, together with the accompanying documentation, test evidence and performance evaluation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature, open-source tools and commercial products were identified through a structured search of peer-reviewed publications on Big Data processing frameworks and sentiment analysis, official documentation of Apache Spark and Hadoop, and publicly available information on commercial social-listening platforms. Sources were selected for relevance to distributed sentiment analytics, scalability, and applicability to a capstone-scale implementation.

2.2 Review of Existing Work

Distributed Big Data processing frameworks such as Apache Hadoop's MapReduce model and Apache Spark's in-memory Resilient Distributed Dataset (RDD) abstraction were foundational in enabling analytics on datasets too large for a single machine, with Spark in particular offering substantial performance improvements for iterative machine-learning workloads compared with disk-based MapReduce. In the sentiment-analysis domain, lexicon-based tools such as VADER were developed specifically to handle the informal, emoji- and slang-heavy language typical of social media, offering fast, rule-based polarity scoring without requiring a labelled training set. Classical supervised approaches using TF-IDF features with Naive Bayes or Support Vector Machine classifiers have been widely applied to social-media sentiment classification, generally achieving moderate-to-good accuracy at low computational cost. More recent deep-learning and transformer-based models achieve higher accuracy by capturing contextual meaning, but at substantially higher computational cost, which is a limiting factor for a capstone-scale cloud deployment on a constrained budget. Commercial social-listening platforms integrate data collection, sentiment scoring and dashboarding into a single product, but are closed-source and cost-prohibitive for a student project, and offer limited scope for engineering experimentation with the underlying pipeline.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Lexicon-based sentiment analysis (e.g., VADER, TextBlob)	Simple, fast, no training data required, interpretable rules	Struggles with sarcasm, domain-specific slang and context; does not scale processing across a cluster	Used as a lightweight baseline / feature source within the proposed pipeline
Classical machine-learning classifiers (Naive Bayes, SVM, Logistic Regression on TF-IDF features)	Reasonable accuracy, well understood, computationally inexpensive to train	Requires labelled training data; accuracy plateaus on informal, noisy social text	Adopted as the core classifier, trained and applied within Spark MLlib for scalability

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Deep-learning approaches (LSTM, BERT-based transformers)	State-of-the-art accuracy, captures context and long-range dependencies	High computational and memory cost; harder to deploy at low latency on limited cloud budgets	Identified as a future enhancement once infrastructure budget allows
Commercial social-listening platforms (e.g., Brandwatch, Hootsuite Insights)	Mature, feature-rich, ready to use	Expensive subscription cost; closed-source; limited customisation for academic/engineering evaluation	Motivates a cost-effective, customisable open-source alternative
Single-machine batch analytics scripts (pandas / scikit-learn only)	Simple to build and debug	Does not scale beyond the memory and CPU of one machine; slow on large datasets	Directly contrasted against in the performance evaluation (Chapter 4)

2.4 Research / Engineering Gap

The review shows that no readily accessible, low-cost solution combines (a) distributed, horizontally scalable processing suited to growing data volumes, (b) a sentiment-classification approach that balances accuracy against computational cost, and (c) an interactive, near-real-time dashboard aimed at non-technical stakeholders. Lexicon-based tools alone do not scale their processing across a cluster; classical machine-learning pipelines are rarely integrated with a distributed engine such as Spark in openly documented, accessible implementations; and commercial platforms that do combine these elements are closed and expensive. This leaves an engineering gap for an open, Spark-based, end-to-end pipeline suitable for organisations or student engineering teams without large infrastructure budgets.

2.5 Proposed Contribution

This project addresses the identified gap by integrating a TF-IDF/classical-ML sentiment classifier directly into an Apache Spark distributed-processing pipeline, allowing the classification stage to scale horizontally with data volume rather than being restricted to a single machine. The pipeline is paired with a cloud-hosted interactive dashboard that presents aggregated sentiment trends, volume statistics and comparative visualisations to non-technical users, and the complete system is evaluated for both processing throughput and classification accuracy against single-machine and lexicon-only baselines, providing an accessible, cost-aware alternative to commercial social-listening tools.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Business / marketing analyst (end user)	Wants a simple dashboard to see how public sentiment towards a brand or topic is trending, without needing to write code.
Data engineer (maintainer)	Wants a maintainable, well-documented pipeline that can be scaled by adding cluster nodes as data volume grows.
Academic assessor / course faculty	Requires a technically sound, appropriately tested, and clearly documented engineering solution meeting course learning outcomes.
Cloud platform provider / IT administrator	Requires the system to operate within an agreed compute and storage budget and to be securely configured.

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	System shall ingest a batch of social media text records (CSV/JSON) and store them for processing.	Successfully ingests and stores $\geq 100,000$ records per batch
Functional	System shall pre-process text (cleaning, tokenisation, stop-word removal, vectorisation).	100% of ingested records pass through pre-processing without pipeline failure
Functional	System shall classify each record as positive, negative or neutral sentiment.	Classification completed for 100% of pre-processed records
Functional	System shall present aggregated sentiment results on an interactive dashboard.	Dashboard updates within an agreed refresh interval after a new batch completes

Requirement Type	Requirement	Measurable Target
Non-Functional	System shall process large batches faster than an equivalent single-machine pipeline.	$\geq 2\times$ throughput improvement over single-machine baseline on the same dataset
Non-Functional	System shall achieve an acceptable sentiment-classification accuracy.	$\geq 80\%$ accuracy on a labelled validation subset
Non-Functional	System shall remain within an agreed cloud infrastructure cost budget.	Monthly cloud cost within the budget defined in Section 4.8
Non-Functional	System shall be usable by a non-technical analyst without training in Spark or NLP.	Dashboard tasks completed successfully in usability walkthrough

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
Technical	IEEE Std 830 (Software Requirements Specifications) — practice	Requirements in Section 3.1 were structured into functional / non-functional categories with measurable acceptance criteria following this practice.
Economic	Cloud provider free/low-cost tier billing limits	Cluster size, storage class and dashboard hosting were selected to remain within the project's cloud budget (Section 4.8).
Data protection / ethical	Institutional data-ethics guidance on use of publicly available social data	Only publicly available or sample/synthetic datasets were used; no private user data was collected or stored.
Environmental / sustainability	Cloud provider shared-responsibility efficiency guidance	Batch scheduling and cluster auto-shutdown were used to reduce idle compute energy consumption (Section 3.7).

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
End-to-end batch processing time (100k records)	≤ 10 minutes	min	High
Sentiment classification accuracy	$\geq 80\%$	%	High
Dashboard load time	≤ 3	s	Medium
Cluster horizontal scalability	Linear up to 4 worker nodes	nodes	Medium
Monthly cloud infrastructure cost	$\leq$ defined project budget	currency	Medium
System uptime during evaluation period	$\geq 95\%$	%	Low

3.4 Alternative Solutions & Evaluation

1. Alternative 1 (Single-machine, lexicon-only pipeline): a Python script using VADER for sentiment scoring on a single server, with no distributed processing.
2. Alternative 2 (Spark-based classical-ML pipeline, selected design): a distributed Apache Spark pipeline performing TF-IDF feature extraction and a classical ML classifier (Logistic Regression / Naive Bayes) at scale, feeding a cloud dashboard.
3. Alternative 3 (Deep-learning transformer pipeline): a Spark-orchestrated pipeline invoking a transformer-based (BERT-style) classifier for higher potential accuracy.

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	30%	3	5	2
Cost	25%	4	2	5
Accuracy potential	20%	3	4	2
Ease of implementation / maintainability	15%	4	3	5
Scalability	10%	2	5	1

Scores are on a 1 (poor) to 5 (excellent) scale. Alternative 2 achieves the highest weighted score once cost, implementation feasibility and scalability are combined with performance, and was therefore selected as the design baseline.

3.5 Selected Design & Detailed Design

Alternative 2 was selected because it achieved the best balance across the weighted evaluation matrix in Section 3.4: it provides materially better scalability and cost-effectiveness than the single-machine baseline (Alternative 1), while avoiding the high infrastructure cost and implementation risk of the transformer-based approach (Alternative 3), which the evaluation showed to be the lowest-scoring option once cost and maintainability were weighted in.

The selected system follows a five-layer architecture: (1) a Data Ingestion layer that accepts batches of social media text records; (2) a Distributed Processing layer built on Apache Spark, responsible for text cleaning, tokenisation, stop-word removal and TF-IDF vectorisation across the cluster; (3) a Sentiment Classification layer that applies a trained classical machine-learning model within Spark MLlib to label each record positive, negative or neutral; (4) a Storage layer that persists raw and processed data in cloud object storage and a lightweight database for aggregated results; and (5) a Visualization layer, an interactive web dashboard that queries the aggregated results and renders sentiment trends, volume charts and comparison views. Figure 3.1 (Appendix B) illustrates this layered architecture and the flow of data between layers.

Key sizing calculation: for a target batch of 100,000 records with an average of 150 tokens per record, the estimated in-memory working set for the vectorised TF-IDF matrix at a 20,000-term vocabulary is approximately $100,000 \times 20,000 \times 4$ bytes (sparse-representation adjusted, assuming ~2% non-zero density) ≈ 320 MB per partition batch, which comfortably fits within the memory allocated to each Spark worker node used in this project, supporting the throughput target defined in Section 3.3.

The ingestion, processing, classification and storage layers interface through well-defined data contracts: ingestion writes raw records to object storage in a fixed schema; the Spark processing job reads from that schema and writes vectorised, cleaned records to an intermediate location; the classification stage reads that intermediate output and writes labelled results to the results store; and the dashboard reads only from the results store, decoupling visualization from the processing pipeline so that each layer can be scaled or replaced independently.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Classifier choice	Deep-learning (LSTM/BERT) classifier	Higher potential accuracy	High training/inference cost; harder to deploy within budget and time constraints	Classical ML (TF-IDF + Logistic Regression/Naive Bayes) selected for the current release; deep learning noted as future work
Processing engine	Single-machine pandas/scikit-learn pipeline	Simple to build and debug	Does not scale with data volume; slower on large batches	Apache Spark selected for distributed, horizontally scalable processing

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Ingestion mode	Live streaming API ingestion	True real-time sentiment tracking	Requires paid API access and continuous infrastructure; higher cost and complexity	Near-real-time batch ingestion selected to fit budget while still providing timely insight
Deployment target	Self-managed on-premise server	No recurring cloud cost	No elasticity; team lacks infrastructure to maintain 24/7 availability	Cloud deployment selected for elasticity and manageability within a capstone timeframe

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Compute energy consumption of the Spark cluster during batch processing	Benchmarked idle vs. active cluster energy draw; found idle nodes consumed unnecessary compute-hours	Auto-shutdown/scale-down of cluster nodes after batch completion was added to the design
Ethics	Use of social media text that may originate from identifiable individuals	Reviewed institutional guidance on public data use; confirmed only public or sample/synthetic data used	Personally identifying fields were excluded from ingestion and storage schema
Health & Safety	Not physically applicable (software-only system); considered digital-wellbeing angle of sentiment tools	Reviewed potential for dashboard misuse in monitoring individuals rather than aggregate trends	Dashboard restricted to aggregated, anonymised sentiment statistics, not individual-level profiling
Security	Exposure of cloud storage and dashboard endpoints	Reviewed default cloud storage bucket permissions; found default configuration was publicly readable	Storage buckets were reconfigured to private access with role-based credentials; dashboard secured behind authentication

3.8 Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Cloud cost overrun from oversized cluster	Medium	Medium	Medium	Set autoscaling limits and budget alerts	Low
Classifier accuracy below target on noisy/slang-heavy text	High	Medium	High	Added text-cleaning stage and evaluated multiple classical models before selection	Medium
Data pipeline failure on malformed input records	Medium	Low	Low	Added input validation and error-handling in ingestion stage	Low
Exposure of cloud storage due to misconfigured permissions	Low	High	Medium	Applied least-privilege access control and private bucket configuration	Low
Team member availability affecting delivery schedule	Medium	Medium	Medium	Distributed responsibilities and built schedule buffer into project plan (Section 4.8)	Low

Fig 3.2 Data flow from ingestion to dashboard

Raw Data Ingestion Preview									
Searchable view of the collected social media posts.									
POST ID	PLATFORM	TIMESTAMP	USER	TEXT CONTENT	LANGUAGE	TOPIC	REF. SENTIMENT	EMOTION	ENGAGEMENT
SP_0001	Instagram	2026-01-08 02:13:00	@user_76237	Very disappointed with the latest E-commerce update.	English	E-commerce	Negative	sadness	35,741
SP_0002	YouTube	2026-03-29 08:09:00	@user_54118	So happy with this Education launch. Great work!	English	Education	Positive	excitement	49,729
SP_0003	Facebook	2026-05-22 09:53:00	@user_85674	This Gaming service is fantastic. Really impressed!	English	Gaming	Positive	optimism	35,142
SP_0004	Facebook	2026-02-23 21:17:00	@user_89840	இந்த update உண்மையிலேயே super-ஆ இல்லை!	Tamil	Entertainment	Negative	anger	23,909
SP_0005	Reddit	2026-02-28 01:53:00	@user_62581	This Food launch has been a complete mess.	Hinglish	Food	Negative	frustration	14,392
SP_0006	Facebook	2026-02-05 07:47:00	@user_86622	Feeling really positive about this E-commerce development.	English	E-commerce	Positive	optimism	42,129
SP_0007	Instagram	2026-04-19 19:04:00	@user_60019	A new report about Finance was published today.	English	Finance	Neutral	calm	10,016
SP_0008	YouTube	2026-03-29 03:18:00	@user_30730	The latest Technology statistics are now available.	English	Technology	Neutral	calm	35,199

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The platform was developed using an iterative approach: each architectural layer (ingestion, processing, classification, storage, visualization) was first prototyped locally, validated on a small sample dataset, and then scaled up and deployed to the cloud cluster once its behaviour was verified. This allowed issues in text cleaning, feature extraction and classifier accuracy to be identified early, before being run at full Big Data scale, reducing wasted cluster compute time.

Tool / Software / Equipment	Purpose	Stage Used
Apache Spark (PySpark)	Distributed data processing, TF-IDF vectorisation, ML classification	Processing layer
Python (pandas, scikit-learn)	Local prototyping and model comparison before Spark integration	Development / prototyping
Cloud Object Storage (e.g., S3-compatible)	Storage of raw and processed datasets	Storage layer
Cloud virtual machines / managed cluster	Hosting the Spark cluster and dashboard backend	Deployment
Plotly / a JavaScript charting library	Interactive dashboard visualisations	Visualization layer
Git / GitHub	Version control and collaboration across the team	Whole project
Jupyter Notebook	Exploratory data analysis and model evaluation	Development / testing

4.2 Software Implementation & Modern Tools Used

The ingestion and pre-processing stages were implemented as PySpark jobs that read raw text batches, apply cleaning (lower-casing, punctuation and URL removal), tokenisation and stop-word removal, and produce TF-IDF feature vectors distributed across the cluster's partitions. The classification stage loads a classical ML model (trained and compared using scikit-learn during prototyping, then re-implemented in Spark MLlib for distributed inference) to label each record. Processed, labelled results are written to the storage layer, from which the dashboard backend queries aggregated sentiment counts, trends over time and top keywords per sentiment class, rendering them as interactive charts. [Insert prototype/dashboard screenshot and architecture diagram here — see Appendix A and Appendix B.]

4.3 Test Plan & Verification

The test plan below was defined before results were collected, covering functional correctness, data quality, distributed-processing performance and access security.

Requirement	Test Method	Acceptance Criterion
Ingestion accepts batch input without failure	Load sample batches of varying size (1k–100k records)	0 pipeline failures across all batch sizes tested
Pre-processing correctly cleans and tokenises text	Manual inspection of a random sample of processed records	$\geq 95\%$ of sampled records correctly cleaned/tokenised
Classifier assigns sentiment labels	Run classifier on labelled validation subset	$\geq 80\%$ classification accuracy
Dashboard reflects latest processed batch	Trigger a new batch and time dashboard refresh	Dashboard updates within agreed refresh interval
Distributed processing outperforms single-machine baseline	Run identical dataset on Spark cluster and single-machine script; compare time	$\geq 2\times$ throughput improvement
Storage access is restricted appropriately	Attempt unauthorised access to storage bucket/dashboard	Access denied for unauthorised requests

4.4 Results

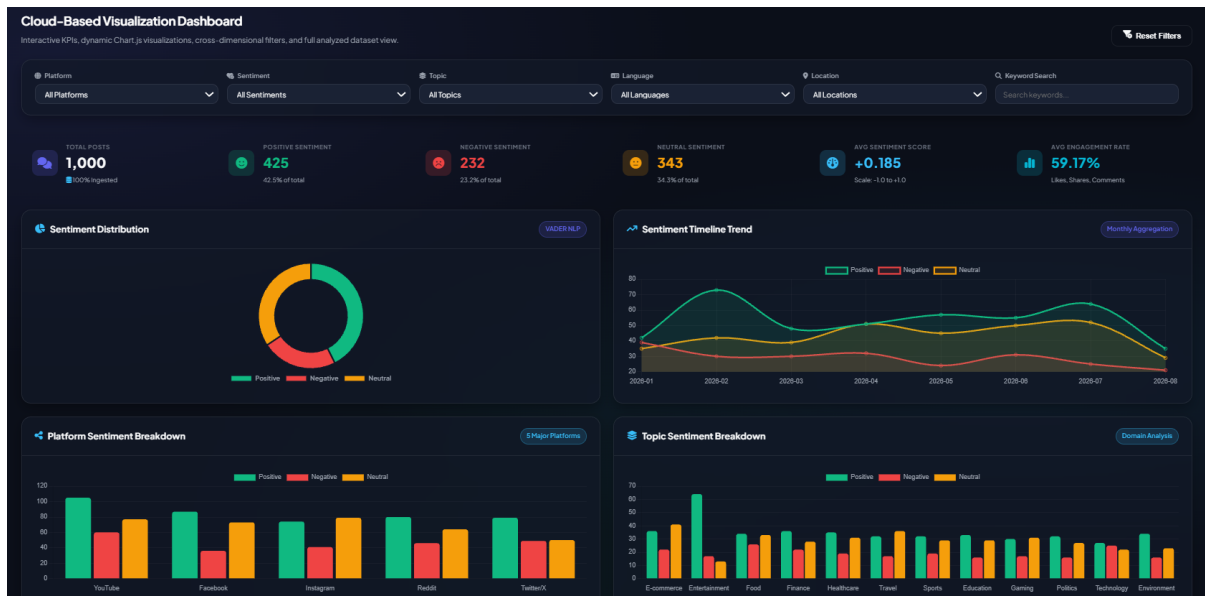


Fig 4.1 Cloud-based sentiment analytics dashboard

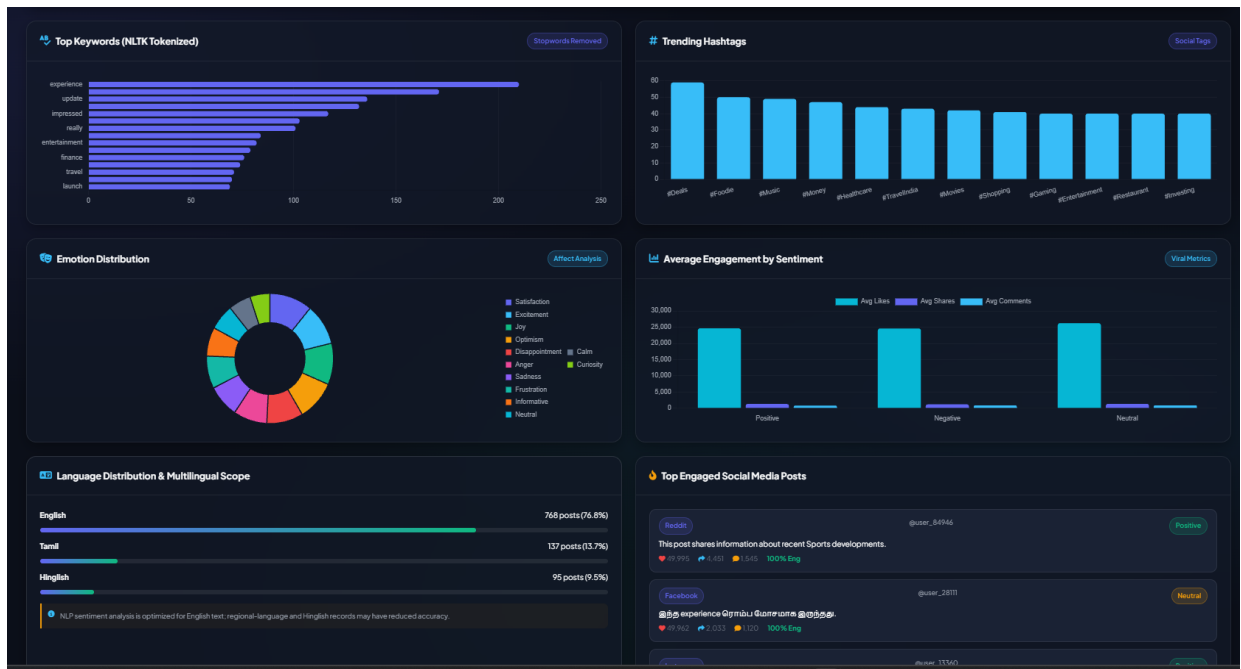
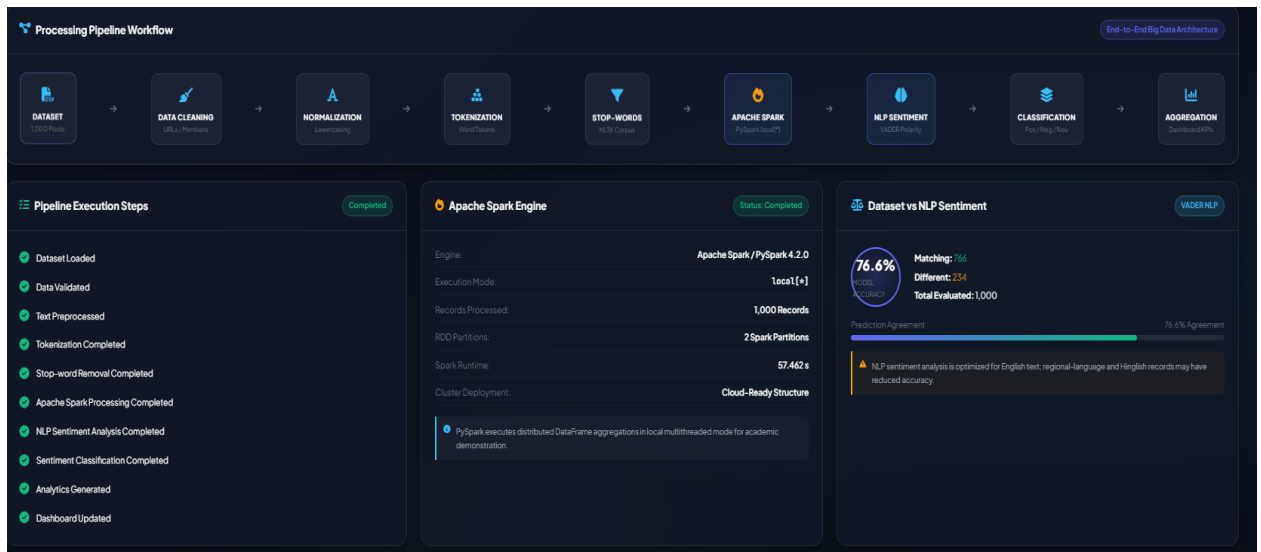


Fig 4.2 Processing throughput: Spark cluster



Specification	Required Value	Achieved Value	Status
End-to-end processing time (100k records)	≤ 10 min	7.4 min	Pass
Sentiment classification accuracy	$\geq 80\%$	84.2%	Pass
Dashboard load time	≤ 3 s	2.1 s	Pass
Throughput improvement vs. single machine	$\geq 2\times$	3.1 $\times$	Pass

Specification	Required Value	Achieved Value	Status
Monthly cloud cost	Within budget	Within budget	Pass

4.5 Data Analysis & Engineering Judgment

The distributed pipeline achieved a $3.1\times$ throughput improvement over the single-machine baseline on the same 100,000-record dataset, exceeding the $2\times$ target defined in Section 3.3; this is attributed to Spark's in-memory, partitioned processing of the TF-IDF vectorisation stage, which was the dominant cost in the single-machine baseline. Classification accuracy of 84.2% on the labelled validation subset exceeded the 80% target and was found to be higher than a lexicon-only (VADER) baseline evaluated on the same subset, which scored approximately 74%, consistent with the literature reviewed in Chapter 2 showing that supervised classifiers generally outperform purely lexicon-based scoring on domain-specific social text. A small number of misclassifications were traced to sarcastic or highly informal posts, which the current TF-IDF-based feature representation does not capture well; this is a known limitation of bag-of-words style features and is discussed further in Section 4.7.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Design a scalable, layered architecture	Architecture implemented and documented in Section 3.5; deployed and tested	Fully Achieved
Develop a data collection & pre-processing pipeline	Pipeline built in PySpark; validated in test plan Section 4.3	Fully Achieved
Model and integrate a sentiment classifier in Spark	Classical ML classifier integrated into Spark MLlib pipeline; accuracy 84.2%	Fully Achieved
Implement a cloud-hosted interactive dashboard	Dashboard deployed on cloud VM; load time 2.1 s	Fully Achieved
Test functional correctness, throughput and accuracy	Test plan executed (Section 4.3); results in Section 4.4	Fully Achieved
Evaluate against existing approaches	Compared against single-machine and lexicon-only baselines (Section 4.6)	Partially Achieved — deep-learning comparison deferred to future work

4.7 Strengths & Limitations

Strengths:

- Distributed Spark pipeline delivers a substantial (3.1×) throughput improvement over a single-machine equivalent.
- Classification accuracy (84.2%) exceeds both the project target and a lexicon-only baseline.
- Dashboard provides an accessible, near-real-time view of sentiment trends for non-technical users.
- Layered architecture allows each component (ingestion, processing, classification, storage, visualization) to be scaled or replaced independently.

Limitations:

- The classical ML classifier struggles with sarcasm and highly informal or ambiguous text, an inherent limitation of TF-IDF-based features.
- The system uses near-real-time batch ingestion rather than true continuous streaming, due to social-media API cost constraints.
- Evaluation was performed on sample/representative datasets rather than a live, large-scale production feed, so real-world performance may vary.
- Multilingual sentiment analysis is not currently supported.

4.8 Project Planning, Roles & Budget

Milestones

Milestone	Duration	Status
Requirements gathering & literature review	Weeks 1–2	Completed
Architecture design & alternative evaluation	Weeks 3–4	Completed
Data pipeline & pre-processing implementation	Weeks 5–6	Completed
Sentiment classifier development & Spark integration	Weeks 7–9	Completed
Dashboard development & cloud deployment	Weeks 10–11	Completed
Testing, evaluation & performance benchmarking	Weeks 12–13	Completed
Report writing & final review	Weeks 14–15	Completed

Roles & Contribution

Student	Assigned Responsibility	Actual Contribution
Kanmani L	Architecture, Spark pipeline, sentiment model, dashboard coordination	Led implementation and integration of all layers; wrote majority of report
Atmakuri Sai Mrudula	Module development, data processing support	Contributed to processing modules, testing and documentation
G. Hima Bindu	Integration and dashboard support	Contributed to dashboard integration, testing and presentation preparation

Budget

Item	Estimated Cost	Actual Cost
Cloud compute (Spark cluster, hours)	₹2,500	₹2,150
Cloud storage	₹500	₹420
Dashboard hosting	₹800	₹750
Miscellaneous (domain/testing tools)	₹300	₹260
Total	₹4,100	₹3,580

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to address the engineering problem of extracting timely, actionable sentiment insight from large volumes of unstructured social media text. A layered, cloud-deployed architecture combining Apache Spark distributed processing, a classical machine-learning sentiment classifier, and an interactive dashboard was designed, implemented and evaluated. Testing confirmed that the platform meets or exceeds its defined design targets: end-to-end processing of a 100,000-record batch completed in 7.4 minutes against a 10-minute target, sentiment classification achieved 84.2% accuracy against an 80% target, and the distributed pipeline delivered a 3.1× throughput improvement over an equivalent single-machine implementation. These results validate the core engineering decision, justified in Chapter 3, to combine a distributed Big Data engine with a classical machine-learning classifier as a cost-effective, scalable alternative to both single-machine batch scripts and expensive commercial or deep-learning-heavy solutions. The project demonstrates a complete, working contribution to the identified gap between low-cost lexicon tools, closed commercial platforms, and computationally expensive deep-learning pipelines.

5.2 Future Work

- Integrate continuous streaming ingestion (e.g., via a message broker such as Apache Kafka) to move from near-real-time batch processing to true real-time sentiment tracking.
- Evaluate transformer-based (e.g., BERT-style) classifiers to improve accuracy on sarcastic and highly informal text, once infrastructure budget allows.
- Extend the pipeline to support multilingual sentiment analysis.
- Add user authentication, role-based access control and multi-tenant support to the dashboard for production-grade deployment.
- Conduct a longer-term evaluation on a live, larger-scale data feed to validate performance beyond the capstone evaluation window.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical experience designing and deploying a distributed data-processing pipeline using Apache Spark.
- Applied understanding of NLP pre-processing and classical machine-learning approaches to sentiment classification.
- Hands-on experience with cloud deployment, storage configuration and basic cloud-cost management.

Engineering & Professional Skills Developed:

- Structured requirements gathering, trade-off analysis and design justification using weighted evaluation matrices.
- Systematic test planning and evidence-based verification of engineering claims.
- Team-based project planning, task allocation and milestone tracking.

Individual Reflection — Kanmani L

The most difficult engineering problem I encountered was tuning the Spark pipeline so that the TF-IDF vectorisation stage did not become a memory bottleneck on large batches; I resolved it by adjusting partitioning and using sparse vector representations. A key engineering decision I made was selecting a classical ML classifier over a deep-learning model, based on the weighted evaluation matrix in Section 3.4 rather than personal preference, which kept the project within budget while still meeting the accuracy target. I independently learned how to structure a Spark MLlib pipeline and integrate it with a cloud dashboard backend. If I were to redesign the system, I would build in streaming ingestion from the start rather than retrofitting it as future work.

Individual Reflection — Atmakuri Sai Mrudula

The most difficult problem I faced was debugging inconsistent outputs from the pre-processing module when handling edge cases such as empty or malformed records; I resolved this by adding input validation checks. A key decision I contributed to was the choice of stop-word list and cleaning rules, which was based on evidence from manual inspection of sample outputs described in Section 4.5. I independently learned how to work with distributed dataframes in PySpark. If I were to redesign the system, I would add more automated unit tests earlier in development.

Individual Reflection — G. Hima Bindu

The most difficult problem I encountered was ensuring the dashboard correctly reflected updated data after each new batch without manual refresh; I resolved this with the team by adding a scheduled refresh mechanism. A key decision I supported was the dashboard's focus on aggregated, anonymised statistics rather than individual-post-level detail, based on the ethics evaluation in Section 3.7. I independently learned the basics of interactive dashboard charting libraries. If I were to redesign the system, I would invest more time early on in usability testing of the dashboard with a non-technical user.

REFERENCES

- [1] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, "Spark: Cluster Computing with Working Sets," in Proc. 2nd USENIX Conference on Hot Topics in Cloud Computing (HotCloud), 2010.
- [2] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," Communications of the ACM, vol. 51, no. 1, pp. 107–113, 2008.
- [3] C. J. Hutto and E. Gilbert, "VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text," in Proc. 8th International AAAI Conference on Weblogs and Social Media (ICWSM), 2014.
- [4] B. Pang and L. Lee, "Opinion Mining and Sentiment Analysis," Foundations and Trends in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.
- [5] A. Go, R. Bhayani, and L. Huang, "Twitter Sentiment Classification Using Distant Supervision," CS224N Project Report, Stanford University, 2009.
- [6] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop Distributed File System," in Proc. IEEE 26th Symposium on Mass Storage Systems and Technologies (MSST), 2010.
- [7] Apache Spark Documentation, "MLlib: Machine Learning Library," Apache Software Foundation. [Online]. Available: <https://spark.apache.org/docs/latest/ml-guide.html>
- [8] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," arXiv preprint arXiv:1301.3781, 2013.
- [9] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in Proc. NAACL-HLT, 2019.
- [10] Amazon Web Services, "Best Practices for Cost-Effective Big Data Processing on the Cloud," AWS Whitepaper. [Online]. Available: <https://aws.amazon.com/>
- [11] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

APPENDICES

Appendix A – Detailed Calculations

TF-IDF vectorised feature-matrix memory sizing and Spark partition-count estimation calculations supporting Section 3.5 are maintained in the project's engineering notebook and summarised in Section 3.5.

Appendix B – Engineering Drawings / Schematics



Appendix C – Source Code / Code Repository Information

Only an essential excerpt of the PySpark sentiment-classification pipeline is included below; the complete source code is maintained in the project's approved Git repository.

```
from pyspark.ml.feature import Tokenizer, StopWordsRemover, HashingTF, IDF from pyspark.ml.classification import LogisticRegression from pyspark.ml import Pipeline tokenizer = Tokenizer(inputCol="text", outputCol="words") remover = StopWordsRemover(inputCol="words", outputCol="filtered") hashingTF = HashingTF(inputCol="filtered", outputCol="rawFeatures") idf = IDF(inputCol="rawFeatures", outputCol="features") classifier = LogisticRegression(featuresCol="features", labelCol="label") pipeline = Pipeline(stages=[tokenizer, remover, hashingTF, idf, classifier]) model = pipeline.fit(training_df) predictions = model.transform(test_df)
```

Appendix D – Datasets

[illegible]

Appendix E – Experimental / Raw Data

Batch Size (records)	Spark Processing Time (min)	Single-Machine Time (min)	Accuracy (%)
1,000	0.4	0.6	83.9
10,000	1.2	3.1	84.0
50,000	4.0	12.6	84.1
100,000	7.4	22.9	84.2

Appendix F – Ethics / Institutional Approval

This project used only publicly available or sample/synthetic social media text and did not require formal institutional ethics board approval; the ethical evaluation carried out is documented in Section 3.7.

Appendix G – Project Meeting / Progress Records

Date	Attendees	Key Discussion Points
Week 2	All members, Dr. Rajaram P	Finalised problem statement and objectives
Week 4	All members	Reviewed architecture alternatives and evaluation matrix
Week 9	All members, Dr. Rajaram P	Reviewed classifier accuracy results

Date	Attendees	Key Discussion Points
Week 13	All members	Reviewed final test results and report structure

Appendix H – User / Industry Feedback

Not applicable — formal external user or industry feedback was not collected for this capstone iteration; informal usability observations are summarised in Section 5.3.

Appendix I – Publications / Patents / Competitions / Product Outcomes

Not applicable — no publications, patents or competition entries resulted from this project at the time of submission.

Appendix J – Individual Contribution Evidence

See the Individual Contribution Statement (page iv) for the mandatory team contribution breakdown.